

SDK / стек

Параметр	Значение
Название приложения	Ice Fishing
Android Gradle Plugin	9.1.1
minSdk	32
targetSdk	36
Kotlin	да, 2.1.0
Web View	да
Custom Tabs	да
Рекламные сети	нет
Аналитика	Adjust, Firebase Messaging
Permissions	android.permission.INTERNET, android.permission.ACCESS_NETWORK_STATE, android.permission.ACCESS_WIFI_STATE, android.permission.VIBRATE, android.permission.POST_NOTIFICATIONS, android.permission.CAMERA, android.permission.WAKE_LOCK, com.google.android.c2dm.permission.RECEIVE, com.frosthunter.arcticwildlands.frozenexpedition.adventure.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.google.android.gms.permission.AD_ID, com.google.android.finsky.permission.BIND_GET_INSTALL_REFERRER_SERVICE, com.android.vending.CHECK_LICENSE
Libraries	Adjust SDK android5.5.0 (com.adjust.sdk, libsigner.so), OkHttp 5.3.2 (WebSocket/HTTP), Coil3, Jetpack Compose UI/Foundation/Animation 1.9.4, Material3 1.3.0, Material Icons/Ripple 1.7.0, AndroidX Activity/Activity-Compose 1.13.0, Fragment 1.8.9, Lifecycle 2.10.0, Core 1.18.0, AndroidX Browser 1.10.0 (Custom Tabs), AndroidX DataStore Preferences 1.2.1, AndroidX Media3, AndroidX Emoji2, Startup, ProfileInstaller, AppCompat Resources, NavigationEvent, kotlinox-coroutines 1.10.2, Firebase Messaging 24.1.2, Firebase Installations, Firebase Common/KTX, Datatransport/CCT, Google Play Services (base/basement/tasks/cloudmessaging/stats), Google Play Install Referrer API, Pairip licensecheck (Play Integrity Protection wrapper), Native libfrost_assemble.so
Подозрительные домены	нет
SharedPreferences	Jetpack DataStore Preferences файл entry_storage.preferences_pb: ключи silentFluxInterpreter (GAID), prismOrbitCoordinator (install referrer), temporalScopeNavigator (FCM token), radialPayloadCompiler (Adjust ID); кэш оффер-URL для повторного входа. Adjust SDK: AdjustIoActivityState / AdjustAttribution / AdjustEventMetadata / AdjustGlobalCallbackParameters / AdjustGlobalPartnerParameters / adjust_keys. Firebase Messaging: com.google.android.gms.appid (FCM token). Оффер-URL также передаётся Intent-extra receiverGameInfoTag.
Есть ли клоака	да
Подозрительные слова	landing.blank, landing.filled, departure.blank, departure.filled, GatherOrder, CargoBox, OpenGlass, OpenReceiver, RetainAndLaunch, receiverGameInfoTag, socket/arrival, screen/departure, transport/ui, WebView, User-Agent, bridge, minLoop, modularContextEmitter, silentFluxInterpreter, prismOrbitCoordinator, temporalScopeNavigator, radialPayloadCompiler

Собираемые данные

- market (GAID / advertisingId; silentFluxInterpreter)
- cloud (FCM token; temporalScopeNavigator)
- source (install referrer; prismOrbitCoordinator)
- adjust (Adjust ID / adid; radialPayloadCompiler)
- campaign (JSON атрибуты Adjust: network / campaign / tracker...)
- packageName
- offer URL (кэш / receiverGameInfoTag)

Как работает клоака

В начале

Снаружи это игра про зимнюю рыбалку (Ice Fishing, package com.frosthunter.arcticwildlands.frozenexpedition.adventure), но при каждом запуске приложение сначала спрашивает у своего скрытого сервера: показать «белую» мини-игру или открыть веб-оффер.

«Нужный» пользователь попадает на http/https-страницу из ответа сервера (конкретный казино/лендинг в APK не зашит — URL приходит снаружи).

Судьбу решает зашифрованный WebSocket-запрос на `wss://zenithpatterncluster.xyz/`: туда уходит пакет `GatherOrder` (см. список выше), а в ответе приходят маршруты `landing.blank` (белая игра) или `landing.filled` (оффер с URL).

Зачем эти данные

`market (GAID)` / `adjust (adid)` → отпечаток устройства и пользователя для фильтрации трафика.

`source (install referrer)` + `campaign` (атрибуция `Adjust`) → источник установки и кампания; от этого зависит оффер или белый путь.

`cloud (FCM token)` → привязка `push`; `packageName` → какая сборка стучится на шлюз.

Куда отправляется запрос

Свой WebSocket-шлюз: `wss://zenithpatterncluster.xyz/` (строка спрятана в `libfrost_assemble.so`, расшифровка XOR-ключом `minLoop`). Пакет `GatherOrder` шифруется PBKDF2-HMAC-SHA256 + AES/GCM/NoPadding с парольной фразой `modularContextEmitter` и уходит как `CargoBox` с метками `transport/ui`.

Это не `Firebase Remote Config`. `Adjust` стартует с токеном `io2wuxomwj4` (тоже из `native`). Если ответ «не тот» / `landing.blank` — открывается `GlassNewActivity` (локальная `Compose/Media3`-игра), без веб-оффера.

Как приходит ответ

Сервер отвечает по маршрутам `socket/arrival` и `screen/departure`. Клиент разбирает развилку:

`landing.blank` → команда `OpenGlass`; `landing.filled` → `RetainAndLaunch(value=URL)` → `OpenReceiver(gameInfo=URL)`.

Решающее поле — наличие URL в `landing.filled`. URL сохраняют в `DataStore` (кэш для повторного входа) и передают в `FrostReceiverActivity` через `Intent-extra receiverGameInfoTag`.

Как показывается целевая страница

Боевой URL открывают во встроенном `WebView` (`FrostReceiverActivity`): `cookies`, `file chooser`, запрос камеры/разрешений; перед `loadUrl` из `User-Agent` убирают маркер «; wv». Параллельно тот же URL умеют открыть через `Chrome Custom Tabs`.

Финальный URL — из ответа WebSocket-шлюза или из локального кэша. Пользователь видит веб-страницу оффера; тип лендинга в коде не зафиксирован.

Цепочка по шагам

Шаг 1. Старт `launcher GeneratorStateActivity` → расшифровка `native`-строк (`Adjust token io2wuxomwj4` и др.) → инициализация `Adjust`.

Шаг 2. Чтение `DataStore entry_storage`: если оффер-URL уже кэширован → сразу `FrostReceiverActivity` с `receiverGameInfoTag`.

Шаг 3. Иначе собирают `GatherOrder`: `market (GAID)`, `cloud (FCM)`, `source (referrer)`, `adjust (adid)`, `campaign` (JSON атрибуции).

Шаг 4. Шифрование (PBKDF2 + AES-GCM, фраза `modularContextEmitter`) → `CargoBox(transport/ui)` → WebSocket `wss://zenithpatterncluster.xyz/`.

Шаг 5. Ответ `socket/arrival` / `screen/departure` → `landing.blank` или `landing.filled` + URL.

Шаг 6. `landing.filled` → `RetainAndLaunch` → `OpenReceiver` → `WebView/Custom Tabs` с URL оффера.

Шаг 7. `landing.blank` / отказ → `OpenGlass` → `GlassNewActivity` (обычная локальная игра).

Шаг 8. Повторный запуск с кэшем URL пропускает новый `probe` и снова открывает оффер.

Роль подозрительных доменов

В таблице отдельных кастомных хостов нет. Роль шлюза решения выполняет `wss://zenithpatterncluster.xyz/` — туда уходит зашифрованный `GatherOrder` и оттуда приходит `landing.blank` / `landing.filled` + URL оффера. Хосты `Adjust/Firebase` — аналитика и `push`, не офферный лендинг.

Финал

«Нужный» пользователь оказывается в `WebView` (или `Custom Tabs`) на внешнем http/https-оффере по URL из ответа `zenithpatterncluster.xyz`. «Ненужный» остаётся в локальной `GlassNewActivity`. Финальный оффер-URL в APK не зашит — его присылает WebSocket-шлюз (или читают из кэша `DataStore`).